

■ 3.7 Linear Algebra

■ 3.7.1 Constructing Matrices

<code>Table[f, {i, m}, {j, n}]</code>	build an $m \times n$ matrix where f is a function of i and j that gives the value of i, j^{th} entry
<code>Array[f, {m, n}]</code>	build an $m \times n$ matrix whose i, j^{th} entry is $f[i, j]$
<code>DiagonalMatrix[list]</code>	generate a diagonal matrix with the elements of <i>list</i> on the diagonal
<code>IdentityMatrix[n]</code>	generate an $n \times n$ identity matrix

Functions for constructing matrices.

This generates a 2×2 matrix whose i, j^{th} entry is $a[i, j]$.

```
In[1]:= Table[a[i, j], {i, 2}, {j, 2}]
Out[1]= {{a[1, 1], a[1, 2]}, {a[2, 1], a[2, 2]}}
```

Here is another way to produce the same matrix.

```
In[2]:= Array[a, {2, 2}]
Out[2]= {{a[1, 1], a[1, 2]}, {a[2, 1], a[2, 2]}}
```

`DiagonalMatrix` makes a matrix with zeroes everywhere except on the leading diagonal.

```
In[3]:= DiagonalMatrix[{a, b, c}]
Out[3]= {{a, 0, 0}, {0, b, 0}, {0, 0, c}}
```

`IdentityMatrix[n]` produces an $n \times n$ identity matrix.

```
In[4]:= IdentityMatrix[3]
Out[4]= {{1, 0, 0}, {0, 1, 0}, {0, 0, 1}}
```

Of the functions for constructing matrices mentioned above, `Table` is the most general. You can use `Table` to produce many kinds of matrices.

<code>Table[0, {m}, {n}]</code>	a zero matrix
<code>Table[Random[], {m}, {n}]</code>	a matrix with random numerical entries
<code>Table[If[i >= j, 1, 0], {i, m}, {j, n}]</code>	a lower triangular matrix
<code>Table[If[i <= j, 1, 0], {i, m}, {j, n}]</code>	an upper triangular matrix
<code>Table[Switch[i-j, -1, a[[i]], 0, b[[i]], 1, c[[i-1]], _, 0], {i, m}, {j, n}]</code>	a tridiagonal matrix

Some special types of matrices.

`Table` evaluates `Random[ ]` separately for each element, to give a different pseudorandom number in each case.

```
In[5]:= Table[Random[ ], {2}, {2}]
Out[5]= {{0.353297, 0.0205034}, {0.117374, 0.546051}}
```

This gives a tridiagonal matrix.

```
In[6]:= Table[Switch[i-j, -1, a, 0, b, 1, c, _, 0],
               {i, 5}, {j, 5}]
Out[6]= {{b, a, 0, 0, 0}, {c, b, a, 0, 0},
          {0, c, b, a, 0}, {0, 0, c, b, a}, {0, 0, 0, c, b}}
```

`MatrixForm` prints the matrix in a two-dimensional array, and makes the structure in this case clearer.

```
In[7]:= MatrixForm[%]
      b a 0 0 0
      c b a 0 0
      0 c b a 0
      0 0 c b a
Out[7]//MatrixForm= 0 0 0 c b
```